

학기 2주차 세션 : RAG & LoRA

📅 날짜	@2026년 3월 12일
🌟 상태	완료

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

1. 어떤 문제를 해결하려 했는가? (Abstract)

기존 LLM은 많은 지식을 파라미터 내부에 저장하지만 아래와 같은 한계점이 존재한다.

1. 지식 접근 및 활용의 한계

- LLM은 모든 지식을 모델 파라미터에 저장해야 한다.
- 특히 **knowledge-intensive task**에서 성능이 제한됨.

2. 지식 업데이트 문제

- 새로운 정보가 생기면 **모델을 다시 학습해야 함**
- 세계 지식(world knowledge)을 지속적으로 업데이트하기 어려움

3. 출처 제공 불가능

- 모델이 어떤 지식을 기반으로 답을 생성했는지 **근거를 제공하기 어려움**

그래서 이 논문에서 RAG를 제안하였다.

- 외부 문서 데이터베이스에서 정보를 검색한 후, 이를 기반으로 답을 생성하는 **retrieval + generation 구조**

2. 연구 동기와 문제점 (Introduction)

LLM은 두 가지 방식으로 지식을 저장한다.

1. Parametric memory

- 지식을 모델 파라미터 내부에 저장

문제점

- 지식 업데이트 어려움
- 저장 용량 제한

- hallucination 발생

2. Non-parametric memory

- 외부 문서 데이터베이스

예시) Wikipedia 문서

장점

- 지식 업데이트 가능
- 확장성 높음

연구 동기

위 두 방식을 결합하면?

⇒ **Hybrid Knowledge Model** = Parametric Memory (LLM) + Non-parametric Memory (Document database)

3. 관련 연구 동향 (Related Work)

1) Single-task Retrieval

retrieval을 특정 NLP task에 적용한 연구

예시) Open-domain QA, Fact Checking, Translation

문제점 : 대부분 **task-specific architecture**

2) General-purpose NLP Architectures

하나의 모델로 여러 NLP task 수행

대표 모델 : BERT, GPT-2, BART, T5

문제점 : 지식이 모두 **모델 파라미터 내부에 저장됨**

3) Learned Retrieval

Neural retrieval 모델 : Dense retrieval, Learned document embedding

4) Memory-based architectures

외부 메모리를 활용하는 모델

5) Retrieve-and-Edit

유사한 데이터를 검색 후 수정하는 방식

예시 : Machine Translation, Semantic Parsing

4. 연구 접근법과 모델 구조 (Method)

쿼리 → 리트리버 → Relevant documents → 제네레이터 → 답

4.1 Retriever

입력 query 에 대해 관련 문서를 검색

top-K 문서 선택

4.2 Generator

retrieved document를 활용하여 답 생성

실험 사용 모델 : BART

5. RAG 구조

RAG에는 두 가지 모델이 있다.

1) RAG-Sequence

전체 sequence 생성 시 **같은 문서 사용**

2) RAG-Token

token 생성 시마다 문서 고려

- RAG-Sequence에 비해 유연하지만, 계산량 증가
-

6. 실험 구성과 결과 (Experiments)

사용 데이터셋 : Natural Questions, WebQuestions, CuratedTREC, TriviaQA, FEVER

성능 비교 모델

- BART
 - DPR
 - T5
-

결과

RAG는 대부분의 **knowledge-intensive task**에서 **SOTA 성능** 달성

특히 Open-domain QA, Fact verification 에서 높은 성능

추가 실험

Knowledge update experiment

→ 문서 index 교체

결과 : 지식 업데이트 가능, 모델 재학습 필요 없음

7. 무엇을 발견했고 한계점은 무엇인가? (Discussion)

1. Retrieval + Generation 결합이 효과적
 2. 외부 지식 활용 가능
 3. knowledge update 가능
-

한계점

1. Retrieval 오류 전파

잘못된 문서 검색 시 → 잘못된 답 생성

2. 계산 비용 증가 → retrieval + generation
 3. 문서 품질 의존
-

8. 결론 및 주요 요약 (Conclusion)

RAG 제안

- external knowledge 활용
- knowledge update 가능
- 여러 NLP task에서 성능 향상

RAG는 **knowledge-intensive NLP task**의 새로운 패러다임을 제시함.

9. 기존 연구 대비 차별점

- 기존
 - retrieval - based QA
 - generative LM
- RAG
 - retrieval + generation

retrieval와 **generation**을 결합하여 하나의 아키텍처로 여러 NLP task 해결

10. 핵심 아이디어

- Parametric Memory (LLM)
+
Non-parametric Memory (Document Index)

외부 문서를 검색한 뒤 → generator가 답 생성

11. 한 줄 핵심 정리

RAG는 retrieval 기반 외부 지식을 generative language model과 결합한 architecture

LoRA : Low-rank adaptation of large language models

• 어떤 문제를 해결하려 했는가? (Abstract)

- 모델의 모든 파라미터를 전체 파인튜닝하게 되면, 미세조정된 모델의 독립적인 인스턴스를 배포하는 것은 엄청나게 비용이 많이 듦, 파인튜닝된 모델의 독립적인 인스턴스를 저장하고 배포하는 것은 매우 어려움.

⇒ Low-Rank Adaptation(LoRA)로 다운스트림 작업에 대한 학습 가능한 파라미터 수를 크게 감소시킴. (훈련 처리량이 높으며 추가적인 추론 지연 없음.)

• 연구 동기와 문제점은 무엇인가? (Introduction)

- 사전 훈련된 모델의 모든 파라미터를 업데이트하려는 파인튜닝을 하려다 보니까 심각한 배포 문제까지 이어지고 있음.
- 하지만, 기존 기법들은 모델 깊이 확장 or 추론 지연 시간을 발생시키거나 모델의 사용 가능한 시퀀스 길이를 줄여서 해결하려고 함.
- 또한, 파인튜닝 기준선에 미치지 못하여 효율성과 모델 품질 간의 트레이드오프가 생김

• 관련 연구 동향은 어떠한가? (Related Works)

- 영감을 얻은 연구 : 학습된 과대적합 모델이 사실 낮은 내재적 차원에 존재함. → 이게 lora의 가정이 됨.
- 전이 학습 : 수십 개의 연구가 모델 적응을 파라미터 및 계산 효율적으로 만들기 위해 노력해옴.

1. 어댑터 레이어 추가

- a. 대규모 신경망은 지연 시간을 낮게 유지하기 위해 하드웨어 병렬 처리에 의존하며 어댑터 레이어는 순차적으로 처리되어야 함 → 매우 작은 병목 차원을 사용하더라도 어댑터를 사용하며 지연 시간이 눈에 띄게 증가함.

Batch Size	32	16	1
Sequence Length Θ	512 0.5M	256 11M	128 11M
Fine-Tune/LoRA	1449.4 $\pm$ 0.8	338.0 $\pm$ 0.6	19.8 $\pm$ 2.7
Adapter ^L	1482.0 $\pm$ 1.0 (+2.2%)	354.8 $\pm$ 0.5 (+5.0%)	23.9 $\pm$ 2.1 (+20.7%)
Adapter ^H	1492.2 $\pm$ 1.0 (+3.0%)	366.3 $\pm$ 0.5 (+8.4%)	25.8 $\pm$ 2.2 (+30.3%)

2. 입력 레이어 활성화의 일부 형태 최적화

- 두 전략 모두 대규모 및 지연 시간에 민감한 프로덕션 시나리오에서는 한계가 있음.

◦ Parameter-Efficient Adaptation

- 많은 연구에서 신경망의 기존 레이어 사이에 어댑터 레이어를 삽입하는 방법을 제안함.
- 어댑터의 최신 확장판 : COMPACTER
 - 본질적으로 일부 미리 결정된 가중치 공유 방식을 사용하여 Kronecker 곱으로 어댑터 레이어를 매개변수화함.

◦ Low-Rank Structures in Deep Learning.

- 이론 문헌에서는 신경망이 다른 고전적인 학습 방법(신경망 탄젠트 커널)보다 우수하며 근본적인 개념 클래스가 특정 저랭크 구조를 가지는 경우 그림.

- [Ghorbani et al., 2020. When do neural networks outperform kernel methods?*](#)
- [O Allen-Zhu & Li2019. What Can ResNet Learn Efficiently, Going Beyond Kernels?](#)
- [Allen-Zhu & Li2020a. Backward Feature Correction: How Deep Learning Performs Deep Learning](#)

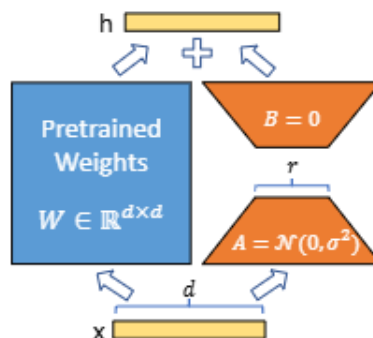
- 또 다른 이론적 결과는 저랭크 적응이 적대적 학습에 유용할 수 있음을 시사함. 신경망이 단순한 커널 근사를 넘어설 수 있는 고유한 학습 능력을 가지고 있음을 보여주며, 이는 LoRA가 대규모 신경망의 특정 부분을 효율적으로 수정함으로써 전체적인 성능을 유지하거나 향상시킬 수 있는 이론적 배경을 제공합니다.

• 연구 접근법과 모델 구조는 어떻게 되는가? (Method)

- 핵심 아이디어 : 신경망은 기본적으로 행렬 곱셈을 수행하는 많은 Dense Layers로 구성됨. 이 레이어들의 가중치 행렬은 일반적으로 'Full-rank'임.

- 기존 연구에 따르면 사전훈련된 모델은 이미지 낮은 내재적 차원을 가지고 있으며 이를 더 작은 부분 공간으로 투영하더라도 효율적으로 학습할 수 있음이 밝혀져있다.

- 위 연구 결과를 바탕으로 본 논문은 모델이 특정 작업에 적응할 때 발생하는 가중치 업데이트 또한 낮은 내재적 랭크를 가질 것이라고 가정함
- 즉, 모델의 모든 가중치가 복잡하게 변하는 것이 아니라, 핵심적인 몇 가지 방향으로만 변화한다는 것 $\Rightarrow$ 이를 통해 학습해야 하는 파라미터 수를 크게 줄일 수 있게 됨.
- 트랜스포머에 LoRA 적용하기
 - 신경망의 가중치 행렬의 임의의 부분집합에 LoRA를 적용하여 학습 가능한 매개변수 수를 줄일 수 있음
 - 트랜스포머 아키텍처 : 셀프 어텐션 모듈 (W_q, W_k, W_v, W_o)에 네 가지 행렬과 MLP 모듈에 두 개의 가중치 행렬이 있음.
 - 다운스트림 작업에 대한 어텐션 가중치만 조정하고 MLP 모듈은 동결함.
- 신경망의 일부 밀집층을 직접 학습하는 대신 사전 학습된 가중치는 고정된 채로 적응 중인 밀집층의 변화에 대한 랭크 분해 행렬을 최적화함으로써 간접적으로 학습할 수 있도록 함.
- 파라미터 효율적인 접근 방식을 채택하며 작업별 파라미터 증분은 훨씬 더 작은 크기의 파라미터 세트에 의해 추가로 인코딩 됨.
- 기존 연구와 차이점은 학습된 가중치가 추론 중에 메인 가중치와 병합될 수 있어 지연 시간을 전혀 발생시키지 않는다는 것임.
- 본 실험에서는 트랜스포머 언어 모델의 특정 가중치에만 초점을 맞춤.



• 실험 구성과 결과는? (Experiments)

- RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), GPT-2 (Radford et al., b)에 대한 LoRA의 다운스트림 작업 성능을 평가한 후 GPT-3 175B (Brown et al., 2020)로 확장
- NVIDIA Tesla V100 사용

1. 베이스라인

2. RoBERTa BASE/LARGE, DeBERTa XXL (GLUE 벤치마크) (*NLU 모델*)

- a. 어댑터와 비교하여 LoRA를 평가하는 방식에 2가지 중요한 변형을 줌.
 - i. 모든 작업에 대해 동일한 배치 크기를 사용하고, 어댑터 베이스라인과 일치 시키기 위해 시퀀스 길이를 128로 사용
 - ii. 파인튜닝 베이스라인과 같이 이미 MNLI에 적응된 모델이 아닌, MRPC, RTE, STS-B에 대해서는 사전 학습된 모델로 모델을 초기화함.

Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 $\pm$ 0	94.2 $\pm$ 1	88.5 $\pm$ 1	60.8 $\pm$ 4	93.1 $\pm$ 1	90.2 $\pm$ 0	71.5 $\pm$ 2	89.7 $\pm$ 3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 $\pm$ 1	94.7 $\pm$ 3	88.4 $\pm$ 1	62.6 $\pm$ 9	93.0 $\pm$ 2	90.6 $\pm$ 0	75.9 $\pm$ 2	90.3 $\pm$ 1	85.4
RoB _{base} (LoRA)	0.3M	87.5 $\pm$ 3	95.1$\pm$2	89.7 $\pm$ 7	63.4 $\pm$ 1	93.3$\pm$3	90.8 $\pm$ 1	86.6$\pm$7	91.5$\pm$2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.9$\pm$1	68.2$\pm$1	94.9$\pm$3	91.6 $\pm$ 1	87.4$\pm$2	92.6$\pm$2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 $\pm$ 3	96.1 $\pm$ 3	90.2 $\pm$ 7	68.3$\pm$1	94.8$\pm$2	91.9$\pm$1	83.8 $\pm$ 2	92.1 $\pm$ 7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5$\pm$3	96.6$\pm$2	89.7 $\pm$ 1	67.8 $\pm$ 2	94.8$\pm$3	91.7 $\pm$ 2	80.1 $\pm$ 2	91.9 $\pm$ 4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 $\pm$ 5	96.2 $\pm$ 3	88.7 $\pm$ 2	66.5 $\pm$ 4	94.7 $\pm$ 2	92.1 $\pm$ 1	83.4 $\pm$ 1	91.0 $\pm$ 1	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 $\pm$ 3	96.3 $\pm$ 5	87.7 $\pm$ 1	66.3 $\pm$ 2	94.7 $\pm$ 2	91.5 $\pm$ 1	72.9 $\pm$ 2	91.5 $\pm$ 5	86.4
RoB _{large} (LoRA)†	0.8M	90.6$\pm$2	96.2 $\pm$ 5	90.2$\pm$1	68.2 $\pm$ 1	94.8$\pm$3	91.6 $\pm$ 2	85.2$\pm$1	92.3$\pm$5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9$\pm$2	96.9 $\pm$ 2	92.6$\pm$6	72.4$\pm$1	96.0$\pm$1	92.9$\pm$1	94.9$\pm$4	93.0$\pm$2	91.3

이 표는 LoRA가 RoBERTa 및 DeBERTa와 같은 대규모 언어 모델에 대해 매우 적은 수의 학습 가능한 파라미터를 사용하면서도, 전체 파인 튜닝(FT)이나 다른 파라미터 효율적 적응 방법론(Adapter, Prefix-tuning 등)과 비슷하거나 더 나은 성능을 보여준다는 것을 입증합니다. 특히, LoRA는 적은 파라미터로도 NLU 작업에서 경쟁력 있는 성능을 달성하며, 이는 모델 배포 및 확장에 큰 이점을 제공합니다.

3. GPT-2 MEDIUM/LARGE (*NLG 모델*)

Model & Method	# Trainable Parameters	E2E NLG Challenge				
		BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4$\pm$.1	8.85$\pm$.02	46.8$\pm$.2	71.8$\pm$.1	2.53$\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49$\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4$\pm$.1	8.89$\pm$.02	46.8$\pm$.2	72.0$\pm$.2	2.47 $\pm$.02

4. , GPT-3 175B로 확장하기

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

GPT-3 175B 모델에 대해 다양한 적응 방법의 성능을 보여줌. LoRA는 전체 파인튜닝을 포함한 다른 적응 방법들보다 더 나은 성과를 보임. WikiSQL 결과는 $\pm 0.5\%$ 내외의 변동을 보이며, MNLI-m은 $\pm 0.1\%$ 내외, SAMSum은 세 가지 지표에 대해 각각 $\pm 0.2/\pm 0.2/\pm 0.1$ 내외의 변동을 보임.

• 무엇을 발견했고 한계점은 무엇인가? (Discussion)

◦ 이점

1. Adam으로 학습된 대규모 트랜스포머의 경우, 동결된 매개변수에 대한 옵티마이저 상태를 저장할 필요가 없으므로 VRAM사용량을 최대 2/3까지 줄임. → 훨씬 적은 수의 GPU로 학습하고 I/O 병목 현상을 피할 수 있음.

2. 모든 매개변수가 아닌 LoRA 가중치만 교체함으로써 배포 시 훨씬 적은 비용으로 작업 간 전환이 가능함. → 사전학습된 가중치를 VRAM에 저장하는 머신에서 즉석에서 전환할 수 있는 많은 맞춤형 모델을 생성할 수 있음

3. 대부분의 매개변수에 대한 그래디언트를 계산할 필요가 없음 → GPT-3 175B에서 전체 파인튜닝 대비 학습 속도가 25% 향상되는 것을 관찰함.

- 한계점

1. 추가적인 추론 지연 시간을 제거하기 위해 A와 B를 W로 흡수하기로 선택한 경우, 단일 순전파에서 서로 다른 A와 B를 가진 다른 작업에 대한 입력을 배치하는 것이 간단하지 않음

- 결론 및 주요 요약은? (Conclusion)

추론 지연을 도입하지 않으면서도 높은 모델 품질을 유지하는 효율적인 적응 전략인 LoRA를 제안. → 모델 파라미터의 대다수를 공유함으로써 서비스로 배포될 때 빠른 작업 전환을 가능하게 함.

논문의 실험에서는 Transformer 언어 모델에 집중했지만, 제안된 원칙은 조밀한 계층을 가진 모든 신경망에 일반적으로 적용될 수 있음.

1. LoRA는 다른 효율적인 적응 방법과 결합될 수 있으며, 잠재적으로 직교적인 개선을 제공할 수 있습니다.
2. 미세 조정 또는 LoRA의 메커니즘은 아직 명확하지 않습니다. 사전 학습 중에 학습된 특징이 다운스트림 작업에서 잘 수행되도록 어떻게 변환되는가?
우리는 LoRA가 전체 미세 조정보다 이 질문에 답하는 것을 더 용이하게 만든다고 믿습니다.
3. 저희는 LoRA를 적용할 가중치 행렬을 선택하기 위해 주로 휴리스틱에 의존합니다
4. $W\Delta$ 의 계수 부족은 W 또한 계수가 부족할 수 있음을 시사하며, 이는 향후 연구에 대한 영감의 원천이 될 수도 있습니다.

- 기존 연구 대비 차별점은 무엇인가?

- 기존 기법들은 모델 깊이 확장 or 추론 지연을 도입하거나 모델의 사용 가능한 시퀀스 길이를 줄여서 해결하려고 함.

- 핵심 아이디어는?

- 사전학습된 모델의 가중치 자체를 직접 업데이트하지 않고, 가중치 업데이트를 저랭크(low-rank) 행렬 분해 형태로 학습하는 것
- 일반적인 파인튜닝에서는 사전학습된 가중치 행렬 W를 직접 업데이트한다.

$$W' = W + \Delta W$$

- 하지만 LoRA에서는 이 가중치 변화 ΔW 가 **낮은 랭크 구조**를 가진다고 가정하고 이를 두 개의 작은 행렬의 곱으로 분해함.

$$\Delta W = BA$$

즉, 전체 가중치 행렬을 업데이트하는 대신 **랭크 rrr 의 작은 행렬 두 개만 학습**하게 된다.
결과적으로 모델의 forward 연산은 다음과 같이 표현됨.